

PDCA TEMPORAL

KairosMix

Sistema de Gestión de Frutos Secos Premium

Ciclo de Mejora Continua

Plan - Do - Check - Act

Tecnologías:

- React 19.1.0
- Vite 7.0.4
- Bootstrap 5.3.7
- ESLint

Documento de calidad - Aseguramiento de Calidad

11 de mayo de 2026

Índice general

Capítulo 1

Introducción y Objetivo General

1.1. Objetivo General del PDCA

Mejorar la calidad, funcionalidad y experiencia de usuario del sistema **KairosMix** mediante un ciclo de mejora continua estructurado bajo la metodología PDCA (Plan-Do-Check-Act).

1.2. Metodología PDCA

El ciclo PDCA es una metodología de mejora continua que consta de cuatro fases:

Plan Planificación de objetivos, alcance y actividades

Do Ejecución de las actividades planificadas

Check Verificación y evaluación de resultados

Act Acciones de mejora y seguimiento

Capítulo 2

PLAN - Planificación

2.1. Objetivos del Ciclo

Los objetivos principales de este ciclo PDCA son:

- Validar la funcionalidad de los módulos implementados
- Identificar defectos y áreas de mejora en el código
- Asegurar calidad en la interfaz de usuario (UI/UX)
- Mejorar la estructura del código y mantenibilidad
- Optimizar el rendimiento de la aplicación web
- Establecer métricas de calidad para futuras iteraciones

2.2. Alcance del Proyecto

El proyecto **KairosMix** es una aplicación web moderna para la gestión integral de un negocio de frutos secos premium. El alcance de este PDCA incluye los siguientes módulos:

2.2.1. Módulo de Gestión de Productos (Implementado)

2.2.2. Módulos en Desarrollo

2.3. Roles y Responsabilidades

2.4. Métricas a Evaluar

Funcionalidad Porcentaje de funciones operativas sin defectos críticos

Defectos Clasificación en Críticos, Mayores y Menores

Cobertura de Pruebas Porcentaje de componentes testeados

Performance Tiempo de carga y respuesta de acciones

Estado	Implementado
Funcionalidades	■ Agregar, editar y eliminar productos ■ Control de stock inteligente con alertas automáticas ■ Categorización por tipo de fruto seco ■ Gestión de proveedores y fechas de vencimiento ■ Búsqueda y filtrado avanzado en tiempo real

Módulo	Estado
Gestión de Clientes	En desarrollo
Gestión de Pedidos	En desarrollo
Mezcla Personalizada	En desarrollo

Accesibilidad Validación de estándares WCAG

Calidad de Código Resultado de ESLint y análisis estático

2.5. Timeline Planificado

Rol	Responsabilidades
Product Owner	▪ Definir requisitos del proyecto ▪ Validar funcionalidad implementada ▪ Priorizar defectos y mejoras
Dev Team	▪ Implementar correcciones ▪ Realizar mejoras técnicas ▪ Optimizar código
QA/Tester	▪ Ejecutar pruebas funcionales ▪ Reportar defectos encontrados ▪ Validar criterios de aceptación
Tech Lead	▪ Supervisar calidad técnica ▪ Revisar arquitectura del código ▪ Definir estándares de desarrollo

Fase	Descripción	Duración
PLAN	Planificación y definición de objetivos	1 día
DO	Ejecución de actividades y desarrollo	3-4 días
CHECK	Verificación y pruebas	2-3 días
ACT	Mejora continua y optimización	2-3 días
Total	Ciclo Completo	1-2 semanas

Capítulo 3

DO - Ejecución

3.1. Actividades Planificadas

3.1.1. A. Desarrollo e Implementación

Completar módulo de Gestión de Clientes

- Formulario de registro de cliente con validación
- Base de datos o almacenamiento local (localStorage/IndexedDB)
- Sistema de búsqueda y filtrado de clientes
- Vista de detalle de cliente

Completar módulo de Gestión de Pedidos

- Interfaz de creación de pedidos
- Asociación de clientes a pedidos
- Cálculo automático de totales y costos
- Historial y consulta de pedidos anteriores

Implementar Mezcla Personalizada

- Selector interactivo de productos
- Cálculo y visualización de información nutricional
- Guardado y recuperación de mezclas personalizadas

3.1.2. B. Testing Inicial

- Pruebas funcionales básicas en cada módulo
- Validación de formularios y datos de entrada
- Pruebas de almacenamiento y persistencia de datos
- Pruebas de integración entre módulos

3.1.3. C. Optimizaciones Técnicas

- Ejecutar análisis de código: `npm run lint`
- Revisar estructura de componentes React
- Optimizar imports y dependencias
- Mejorar performance de renderizado

3.2. Checklist de Implementación

Antes de pasar a la fase CHECK, validar:

Item	Estado
Todos los módulos funcionan en el navegador	
No hay errores en la consola del navegador	
ESLint no reporta errores críticos	
Los datos persisten correctamente	
Responsive design funciona en móvil y desktop	
Aplicación puede ser compilada sin errores	
Documentación básica completada	

Capítulo 4

CHECK - Verificación

4.1. Plan de Testing

4.1.1. Pruebas Funcionales

Módulo de Productos

- Crear un producto nuevo con validación de campos
- Editar información de un producto existente
- Eliminar un producto del sistema
- Aplicar filtros de búsqueda en tiempo real
- Validar alertas automáticas de stock bajo

Módulo de Clientes

- Registrar nuevo cliente con datos completos
- Editar información de cliente
- Buscar cliente por nombre o email
- Validar información obligatoria
- Eliminar cliente del sistema

Módulo de Pedidos

- Crear nuevo pedido asociado a un cliente
- Agregar productos a un pedido
- Calcular total correctamente (incluyen impuestos si aplica)
- Generar reporte de pedidos
- Editar y cancelar pedidos

Mezcla Personalizada

- Seleccionar múltiples productos
- Ver información nutricional combinada
- Guardar mezcla personalizada
- Recuperar mezclas guardadas previamente
- Editar mezclas existentes

4.1.2. Pruebas de Calidad

UI/UX

Criterio	Validado
Consistencia visual en toda la aplicación	
Paleta de colores (café y verde) coherente	
Accesibilidad: contraste de colores (WCAG)	
Botones e iconos bien identificados	
Mensajes de error claros y útiles	
Responsive en dispositivos móviles	
Formularios intuitivos y fáciles de usar	

Rendimiento

- Tiempo de carga inicial < 3 segundos
- No hay lag perceptible en interacciones
- Gestión eficiente de memoria (sin memory leaks)
- Aplicación funciona bien en conexiones lentas

Código

Criterio	Cumple
Sin errores de ESLint	
Componentes reutilizables	
Props validadas correctamente	
Estructura clara de carpetas	
Nombres de variables descriptivos	
Código comentado donde sea necesario	

Campo	Descripción
ID	Código único del defecto (ej: DEFECTO-001)
Severidad	Crítico Mayor Menor
Módulo	Nombre del módulo afectado
Descripción	Descripción clara del problema
Pasos para reproducir	Lista detallada 1. 2. 3.
Resultado esperado	Comportamiento correcto
Resultado actual	Comportamiento observado
Asignado a	Miembro responsable
Estado	Abierto En progreso Resuelto

4.2. Reporte de Defectos

4.2.1. Plantilla de Defecto

4.3. Matriz de Evaluación

Criterio	Peso	Estado	Observaciones
Funcionalidad	40 %		
Calidad de Código	25 %		
UX/UI	20 %		
Rendimiento	10 %		
Testing Coverage	5 %		
TOTAL	100 %		

Leyenda: Aprobado | Parcial | No aprobado | Por evaluar

4.4. Resultados Esperados

Los criterios de aceptación para esta fase CHECK son:

- Mínimo 85 % de funcionalidad operativa
- 0 defectos críticos identificados
- Máximo 5 defectos mayores
- ESLint sin errores o solo warnings menores
- Score de accesibilidad ≥ 80
- Tiempo de carga < 3 segundos

Capítulo 5

ACT - Acciones de Mejora

5.1. Análisis de Resultados

5.1.1. Actividades

- Documentar hallazgos principales del ciclo
- Priorizar defectos encontrados por severidad
- Identificar raíces de problemas comunes
- Clasificar mejoras por impacto y complejidad
- Generar lecciones aprendidas

5.2. Plan de Mejora Continua

5.2.1. Mejoras Inmediatas (Sprint Siguiente)

- Corregir todos los defectos críticos identificados
- Optimizar componentes con bajo rendimiento
- Mejorar mensajes de error y validaciones de formularios
- Implementar confirmaciones para acciones destructivas

5.2.2. Mejoras a Corto Plazo (2-4 semanas)

- Implementar pruebas unitarias con Jest
- Agregar logging y monitoreo en consola
- Documentar API interna del proyecto
- Mejorar accesibilidad a nivel WCAG AA
- Crear guía de estilo de componentes

5.2.3. Mejoras a Largo Plazo (1-2 meses)

- Integración con backend real (Node.js/Express)
- Sistema de autenticación robusto con JWT
- Base de datos SQL o NoSQL
- Análisis y reportes avanzados
- Exportación a Excel mejorada con formatos
- Sistema de notificaciones en tiempo real

5.3. Acciones de Seguimiento

Acción	Responsable	Fecha	Dependencias	Estado

5.4. Plan para Próximo Ciclo PDCA

- Definir nuevos objetivos de mejora basados en resultados
- Ajustar métricas de evaluación si es necesario
- Incorporar feedback de usuarios finales
- Planificar incremento de funcionalidades
- Mantener registro histórico de mejoras

Capítulo 6

Indicadores Clave (KPIs)

6.1. Seguimiento de Métricas

KPI	Meta	Actual	Estado
% Funcionalidad Operativa	85 %		
Defectos Críticos	0		
Defectos Mayores	≤ 5		
Tiempo Promedio de Carga	$< 3s$		
Cobertura de Testing	$\geq 50 \%$		
Score ESLint	100 %		
Accesibilidad (WCAG)	≥ 80		

Capítulo 7

Información del Equipo

7.1. Miembros del Proyecto

Nombre	Rol	Email
	Product Owner	
	Tech Lead	
	Developer 1	
	Developer 2	
	QA/Tester	

7.2. Escalaciones y Contacto

Temas Técnicos Contactar al Tech Lead

Efectos Críticos Revisar e implementar corrección inmediata

Cambios de Scope Consultar con Product Owner y replanificar

Bloqueos Escalar a Tech Lead para desbloqueo urgente

Capítulo 8

Recursos Técnicos

8.1. Stack Tecnológico

Componente	Tecnología
Frontend Framework	React 19.1.0
Build Tool	Vite 7.0.4
CSS Framework	Bootstrap 5.3.7
Iconografía	Lucide React
Routing	React Router v7
Exportación	XLSX (Excel)
Alerts	SweetAlert2 11.22.2
Linting	ESLint 9.30.1

8.2. Documentación de Referencia

- Documentación Oficial React
- Documentación Vite
- Bootstrap 5 Documentation
- ESLint Rules
- WCAG 2.1 Accessibility Guidelines

Registro de Cambios

Ciclo #1

Fase	Resultado	Notas
PLAN	Completado	Objetivos y alcance claros definidos
DO	En Progreso	Implementación en curso
CHECK	Pendiente	Próxima fase a ejecutar
ACT	Pendiente	Acciones de mejora pendientes

Conclusión

Este documento **PDCA Temporal** proporciona un marco estructurado para la mejora continua del proyecto **KairosMix**. Mediante la aplicación rigurosa de las fases Plan-Do-Check-Act, el equipo puede:

- Identificar y resolver defectos de manera sistemática
- Establecer estándares de calidad claros
- Mejorar continuamente la arquitectura y código
- Optimizar la experiencia del usuario
- Documentar lecciones aprendidas
- Preparar la base para ciclos PDCA futuros

Última actualización: 11 de mayo de 2026

Próxima revisión: Una semana después de iniciado el ciclo

Este es un documento dinámico de Aseguramiento de Calidad que debe ser actualizado continuamente según los avances del proyecto.